



4multithread-programming

Process Öldürürken Kullanılan Sinyaller

Bir süreci `kill()` komutuyla öldürebileceğimizi biliyoruz. Ancak, öldürme sinyallerinin (signals) iki önemli çeşidi vardır:

1. **SIGTERM (Kibar Ölüm):** En yaygın kullanılan kapatma sinyalidir. İşletim sistemi sürece "Kapan" der, ancak süreç isterse bu sinyali yakalayıp son bir temizlik (dosyaları kaydetme vb.) yapıp öyle kapanabilir.
2. **SIGKILL (Acımasız Ölüm):** Bu çok daha güçlü bir sinyaldir. Sürece hiçbir söz hakkı tanımaz, temizlik yapmasına izin vermeden süreci anında ve kesin olarak öldürür.

Yeni Süreç Yükleme Varyasyonları

- **execve** : Programın tam dosya yolunu (pathname) ve ortam değişkenlerini (environment variables) senin vermen gerekir.
- **execvp** : Bu çok akıllıdır. Sen sadece programın adını yazarsın, o gidip sistemdeki **PATH** değişkenine (klasör yollarına) bakarak programın nerede olduğunu kendisi bulur.
- **execlp** : Programın parametrelerini toplu bir liste (dizi) olarak değil de, tek tek, aralarına virgül koyarak vermeni sağlar.

Pipes

- İşletim sistemi bir boru (pipe) yarattığında sana iki tane File Descriptor numarası verir.
- Biri okuma ucu, diğeri yazma ucudur. **Kural şudur:** Veriler her zaman `fds[1]` numaralı uca yazılır ve `fds[0]` numaralı uçtan okunur.

- Eğer yazma ucu (`fds[1]`) tamamen kapatılırsa, okuyan taraf dosyanın sonuna geldiğini (EOF - End of File) anlar.
-

Çok Çekirdekli Sistemlerin Zorlukları

- **Aktiviteleri Bölmek:** Büyük bir işi, alt parçalara mantıklı bir şekilde ayırmak.
- **Denge (Balance):** Çekirdeklerin bir kısmı boş yatarken diğerlerinin aşırı yüklenmesini engellemek.
- **Veri Bölme (Data splitting):** İşlenen veriyi çekirdeklere paylaşmak.
- **Veri Bağımlılığı (Data dependency):** Bir çekirdeğin yapacağı işin, diğer çekirdeğin üreteceği sonuca bağlı olması durumu.
- **Test ve Hata Ayıklama (Debugging):** Çok çekirdekli kodlarda hata bulmak, normal kodlara göre çok daha karmaşıktır.

Eşzamanlılık ve Paralellik

Eşzamanlılık

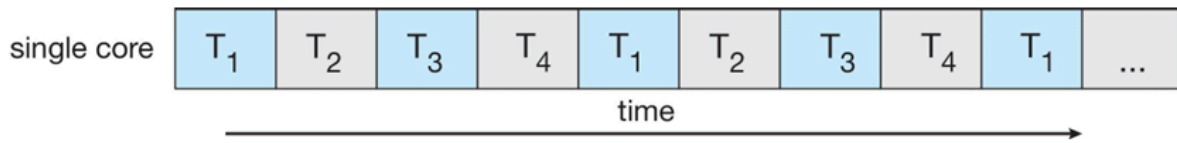
Birden fazla görevin bağımsız olarak ilerleme kaydedebilmesidir.

Tek çekirdekli bir sistemde bile, işletim sistemi görevler arasında çok hızlı geçiş yaparak (zamanlayıcı sayesinde) hepsi aynı anda çalışıyormuş illüzyonu yaratabilir.

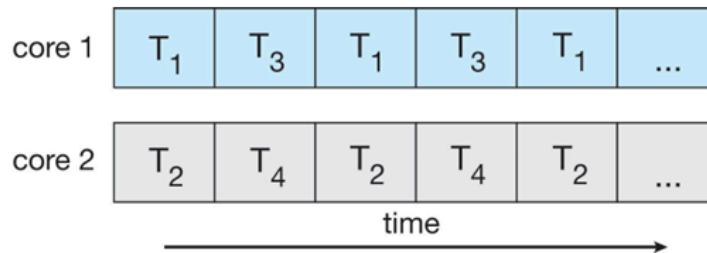
Paralellik

Sistemin, birden fazla görevi **fiziksel olarak tam anlamıyla aynı anda** yapabilmesidir. Bunun için çok çekirdekli (multi-core) bir donanım şarttır.

■ Concurrent execution on single-core system:

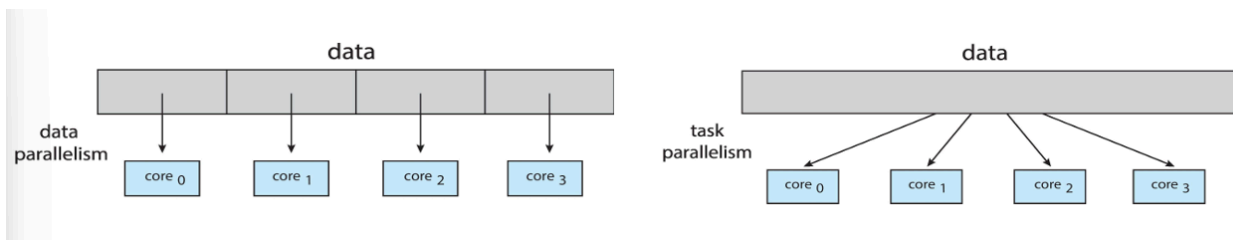


■ Parallelism on a multi-core system:



Paralellik Türleri

- **Veri Paralelliği:** Aynı veri kümesi alt parçalara bölünür ve farklı çekirdeklere dağıtılır. Tüm çekirdekler aynı işlemi yapar. (Örnek: Matris toplama işleminde, herkes kendi payına düşen sayıları toplar ``).
- **Görev Paralelliği:** Veri aynı olsa da, farklı iş parçacıkları (threads) farklı çekirdeklere dağıtılır ve her biri **tamamen farklı ve benzersiz bir işlem** yapar. (Örnek: Aynı veri üzerinden hem toplama hem de çarpma işleminin aynı anda farklı çekirdeklerde yapılması).



Amdahl Yasası

Bir programa sınırsız sayıda çekirdek eklersen, o program sınırsız derecede hızlanır mı? Amdahl Yasası kesin bir dille "**Hayır!**" der.

- Formül: $Hızlanma \leq \frac{1}{S + \frac{(1-S)}{N}}$.
- Buradaki S , programın doğası gereği mecburen **sırayla** çalışması gereken yüzdesidir. N ise çekirdek sayısıdır.
- **Yasanın Mantiği:** Performans kazancını belirleyen şey, koda eklediğin çekirdek sayısı değil, kodun seri çalışmak zorunda olan o aşılmaz bariyeridir.
- Eğer bir programın %25'i mecburen tek çekirdekte sırayla çalışmak zorundaysa (örneğin diğer verileri beklediği için), sisteme istersen 1 milyon tane çekirdek ($N \rightarrow \infty$) ekle, programın ulaşabileceği **maksimum hızlanma teorik olarak 4 katı geçemez.**

Thread Bellek Mimarisi

Bir süreç (process) içinde birden fazla iş parçacığı (thread) yaratıldığında, bu parçacıklar aynı çatı altındadır ama her şeylerini paylaşmazlar.

- **Neleri Paylaşırlar (Ortak)?** Kod (text) bölümü, global veriler (data), heap (dinamik bellek), açık dosyalar, soketler ve süreç kimlikleri (PID, User ID vb.) tüm thread'ler arasında **ortaktır**.
- **Neleri Paylaşmazlar (Özel/Kendine Has)?** Her thread'in nerede kaldığını tutan Program Sayacı, donanım yazmaçları (Registers) ve fonksiyonların çağrıldığı Yığıt (Stack) alanı her thread için **bambaşka ve özeldir**.

Thread'lerin Avantajları

- **Tepkisellik (Responsiveness):** Programın bir kısmı takılsa (bloklansa) veya ağır bir işlem yapsa bile, diğer thread'ler sayesinde kullanıcı arayüzü donmaz ve çalışmaya devam eder.
- **Kaynak Paylaşımı (Resource Sharing):** Thread'ler aynı belleği paylaştığı için birbirleriyle haberleşmeleri çok daha kolaydır.
- **Ekonomi (Economy):** Yeni bir process yaratmak (fork) işletim sistemi için çok ağırdır. Thread yaratmak ve thread'ler arası geçiş (context switch) yapmak

bellek açısından çok daha ucuz ve düşük maliyetlidir.

- **Ölçeklenebilirlik (Scalability):** Modern çok çekirdekli mimarilerin gücünden tam anlamıyla faydalanmayı sağlar.

Thread Senkronizasyonu

Hoca matris çarpımı örneğinde çok önemli bir teorik kuralı vurgulamış: **Ne zaman senkronizasyona (kilitlere/lock) ihtiyacımız olmaz?**

- Eğer birden fazla thread aynı veri kümesini sadece **okuyorsa** (hiçbiri veriyi değiştirmiyorsa) veya
- Her thread bellekte **tamamen birbirinden bağımsız, farklı adreslere/satırlara yazma işlemi yapıyorsa**,
- O zaman aralarında hiçbir yarış durumu (Race Condition) oluşmaz. Kilit (Lock/Mutex) kullanmana gerek kalmaz. Buna mimaride **Thread Safety by Design (Tasarımla Gelen İş Parçacığı Güvenliği)** denir.

User and Kernel Threads

Kernel Threads

Preemptive & Non-Preemptive Threads

- **Non-preemptive Threads (Kesintisiz İş Parçacıkları):** Bir iş parçacığı işlemciyi ele geçirdiğinde, kendi isteğiyle durana veya bir bekleme (blocking) çağrısı yapana kadar **işlemciyi tamamen kendine özel (exclusively) olarak kullanmaya devam eder**. İşletim sistemi onu zorla durdurup işlemciyi elinden almaz.
- **Preemptive Threads (Kesintili İş Parçacıkları):** İşletim sistemleri dersinde genellikle standart olarak kabul ettiğimiz modern model budur. Bu modelde sistem, çalışan bir iş parçacığını herhangi bir anda, thread'i durdurup işlemciyi sıradaki bir başka thread'e verebilir (switch) .

Kernel Seviyesinde Thread Yaratma

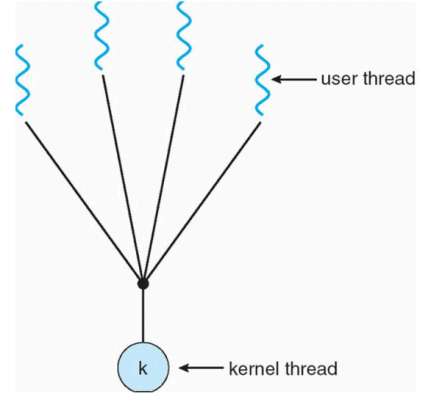
- Programında thread yarat dediğinde, aslında arka planda işletim sistemine bir **Sistem Çağrısı** yapmış olursun.
- İşletim sistemi bu çağrıyı alır, yeni bir süreç yaratıyormuş gibi davranır ama bunu yaparken bellek alanını (address space) ve açık dosyaları kopyalamak yerine **mevcut olanları paylaşır**.
- Bu işlem, sıfırdan bir süreç (process) yaratmaktan çok daha hızlıdır, ancak yine de oldukça **ağır bir işlemdir (heavy weight)**.

Çekirdek Seviyesi Thread'lerin Dezavantajları

- **Maliyet ve Yavaşlık:** Bir Kernel Thread yaratmak, sonlandırmak, bekletmek (join) veya aralarında geçiş yapmak (context switch) için **her seferinde Kernel'a (İşletim Sistemine) gitmek zorundasın**.
 - Normal bir fonksiyon çağırmak işlemcide 5 döngü (cycle) sürerken, Kernel'a geçiş yapmak (syscall) 100 döngü sürer . Bu yüzden Kernel Thread'ler **10 ila 30 kat daha yavaştır**.
- **Esneklik Eksikliği:** İşletim sistemi, thread yönetimini "herkese uyacak" genel bir standartta tasarlar.
 - Senin programının o kadar karmaşık bir önceliklendirmeye veya özelliğe ihtiyacı olmasa bile, Kernel'ın sunduğu o ağır yapıyı kullanmak ve bedelini ödemek zorunda kalırsın .
- **Hafıza Tüketimi:** Her bir Kernel Thread, çekirdeğin içinde kendisine ait sabit boyutlu bir yığıt (stack) ve çeşitli veri yapıları gerektirir. Yüz binlerce thread açmaya kalkarsan Kernel'ın hafızası şişer.

User Threads

- Bu yapılara genellikle "Green Threads" veya "Fibers" adı verilir.
- Sistemde yaratılan tüm bu kullanıcı seviyesi iş parçacıkları, işletim sisteminin gözünde sadece **tek bir Kernel Thread** olarak görünür ve onun üzerinde çalışır.



User Thread'lerinin Yönetilmesi

User thread'ler kernelde tek thread olarak göründükleri için yönetimleri kernel tarafından yapılmaz, bu işlemi bir kütüphane üstlenir.

- **Bellek Yönetimi:** Kütüphane, `thread_create` her çağrıldığında o thread için bellekte (heap üzerinde) yeni ve özel bir yığıt (stack) alanı tahsis eder.
- **Kuyruk Yönetimi:** Çalışmaya hazır (runnable) olan thread'leri bir kuyruktaki (queue) tutar.
- **Sistem Çağrılarını Kandırmak (Fake I/O):** Eğer bir thread ağdan veri okumak gibi normalde sistemi donduracak (blocking) bir işlem yaparsa, kütüphane bu çağrıyı yakalar.
 - İşletim sistemine "Beklemesiz (Non-blocking)" bir okuma emri gönderir.
 - Veri gelene kadar o thread'i uyutur ve kuyruktaki diğer thread'i çalıştırır.
- **Zorla Kesme (Preemption):** Kütüphane, thread'lerin işlemciyi sonsuza kadar rehin almasını engellemek için işletim sisteminden periyodik bir zamanlayıcı sinyali kurar. Sinyal geldiğinde kütüphane çalışan thread'i zorla durdurup diğerine geçer.

User Thread İçi Register Paylaşımı

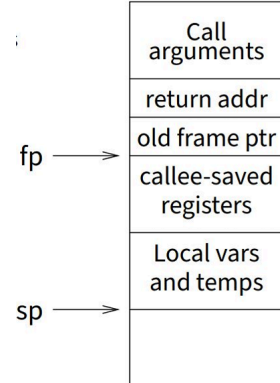
CPU çekirdeğindeki register'lar çok hızlı olmalarına karşın çok azlardır ve tek bir thread içinde çalışan farklı fonksiyonlar bunları kendi aralarında paylaşmak zorundadır.

A Fonksiyonu (Çağırıcı - Caller) masada (register) çalışırken, bir alt iş için **B Fonksiyonunu** (Çağrılan - Callee) göreve çağırır ama masa ortaktır. B Fonksiyonu masaya gelip A'nın verilerini silip kendi işini yaparsa, A geri döndüğünde çalışamaz ve program çöker.

Bu karmaşanın önlenmesi için kibarlık kuralları yazılmıştır:

1. Çağırıcının Sorumluluğundaki Yazmaçlar (Caller-saved):

- Bu yazmaçlar `%eax`, `%edx`, `%ecx` 'dir. Bunlar Çağırıcının sorumluluğundadır ve çağrılan bunları istediği gibi dağırabilir.
- Mantığı:* A Fonksiyonu, B'yi çağırmadan önce sadece *gerçekten* işi bitmemiş ve B'den döndükten sonra *kesinlikle* kullanması gereken hayati bir verisi varsa bunları yığına kaydeder.



1. Çağrılanın Sorumluluğundaki Yazmaçlar (Callee-saved):

- Bu yazmaçlar `%ebx`, `%esi`, `%edi`, `%ebp`, `%esp` 'dir. Bunlar çağrılanın sorumluluğundadır ve çağırıcı buralardaki önemli bilgiler stack'e push etmez.
- Mantığı:* B Fonksiyonu çalışmaya başladığında şöyle der: *"A beni çağırdı. Masadaki bu özel yazmaçları kullanmam lazım ama işim bittiğinde, A'ya geri dönmenden önce bunları **tam olarak bulduğum gibi (eski haline getirerek)** bırakacağıma söz veriyorum."*
- Bu yüzden B, bu yazmaçları değiştirmeden önce kendi çantasına (Yığıt - Stack) kaydeder ve çıkarken geri yükler.



Bunlar aslında kernel seviyesindeki işlemlerdir, user-level'de aynısı kütüphaneler aracılığı ile yapılabilir.

User-Level Process'lerin Avantajları ve Dezavantajları

1. Avantajları:

- a. Aynı süreç içindeki kullanıcı thread'leri arasında context switch yapmak inanılmaz hızlıdır. Çünkü kernel mode'a geçiş gibi ağır bir yük yoktur. Sadece register'leri kaydet ve diğerini yükle mantığında çalışır.

2. Dezavantajları:

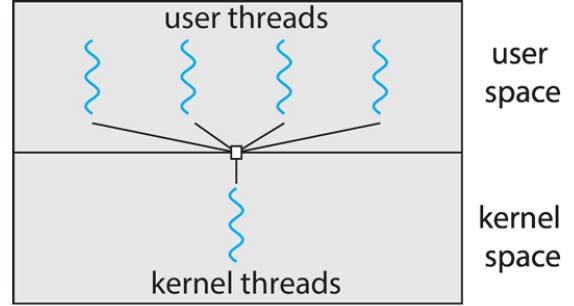
- a. Çoklu çekirdek gücünden faydalanamaz:
 - i. İşletim sistemini gözünde tek bir kernel thread vardır.
 - ii. Sen içeride kütüphane ile 10.000 tane kullanıcı thread'i yaratsan bile, işletim sistemi bunları alıp 4 farklı çekirdeğe dağıtamaz. Hepsi tek bir çekirdekte sırasını beklemeye mahkumdur.
- b. Bir thread'in blocking yapması diğer bütün thread'lere engel olur:
 - i. Eğer thread'lerinden biri, işletim sistemini bekletmek zorunda kalacağı bir işlem (blocking) yaparsa, işletim sistemi o sürecin tamamını uyutur. İçeride çalışmaya hazır bekleyen diğer tüm kullanıcı thread'leri de haksız yere bloke olmuş olur.
 - ii. Ağ işlemlerinde `O_NONBLOCK` (beklemesiz) komutu kullanılarak bu donma engellenebilir.
 - iii. Ancak önbelleğe alınmamış bir disk okumasında veya senkronize disk yazmalarında kaçış yoktur.
- c. Kilitlenme (Deadlock) Riski:
 - i. Eğer bir kullanıcı thread'i diğer bir thread'i beklemeye başlarsa, tüm süreç tıkanabilir ve program hiçbir ilerleme kaydedemez.

Multi-Threading Models

Mapping: User space'te yaratılan thread'lerin işlemcide çalışabilmeleri için kernel space thread'lerle eşleştirilmesi gerekir. Aşağıdaki modellerin hepsi user-level thread'leri kernel-level thread'lerle map eder.

Many to One Model

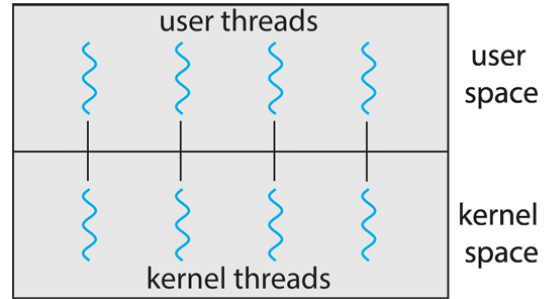
- Kullanıcı kütüphanesinin yarattığı çok sayıdaki iş parçacığının hepsi, arka planda sadece **tek bir** Kernel Thread'e bağlanır.
- Sistemde birden fazla çekirdek (multicore) olsa bile, hepsi tek bir Kernel Thread'e bağlı olduğu için aynı anda paralel çalışamazlar.



- User-Thread'lerin dezavantajları başlığındaki olumsuzlukların hepsi geçerlidir.

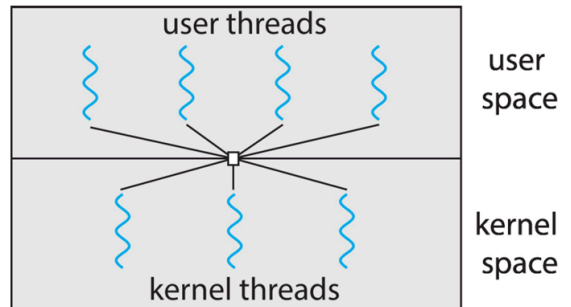
One to One Model

- Yarattığın her bir Kullanıcı Thread'i için, işletim sistemi arka planda özel bir Kernel Thread yaratır.
- Eşzamanlılık (Concurrency) sağlar.
- Her yeni kullanıcı thread'i için Kernel'a inip bir yapı kurmak overhead yaratır, bu yüzden süreç başına yaratılabilecek thread sayısına kısıtlama getirmek gerekebilir.



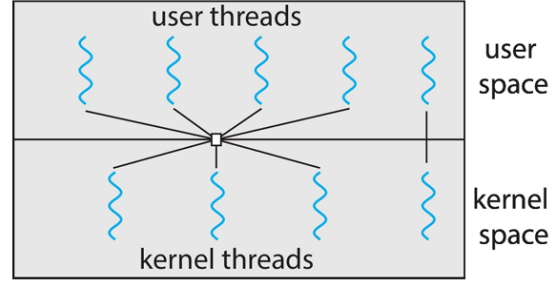
Many to Many Model

- Çok sayıdaki Kullanıcı Thread'i, kendilerinden sayıca eşit veya daha az olan Kernel Thread havuzuna bağlanır.



İki Seviyeli Model

- Many to many modele çok benzerdir. Sadece farklı olarak programcının kritik gördüğü bir kullanıcı thread'ini kalıcı olarak tek bir kernel thread'e bağlamasına izin verir.



Many to Many Modelinin Patladığı Yerler

- İşletim sistemi (Kernel) sistemdeki çekirdek sayısını ve hangi Kernel Thread'in bloke olduğunu bilir. Ancak güvenlik için bunları uygulamadan gizlemeye çalışır. Bu yüzden kullanıcı seviyesindeki zamanlayıcı (scheduler) kendi thread'inin tıkr tıkr çalıştığını sanırken, altındaki Kernel Thread aslında bloke olmuş olabilir.
- İşletim sistemi (Kernel), senin thread'lerinin hangisinin daha önemli olduğunu bilmez. Örneğin, kütüphane için çok önemli bir kilidi elinde tutan bir thread'in altındaki Kernel Thread'i süresi doldu diyerek acımasızca kesebilir.

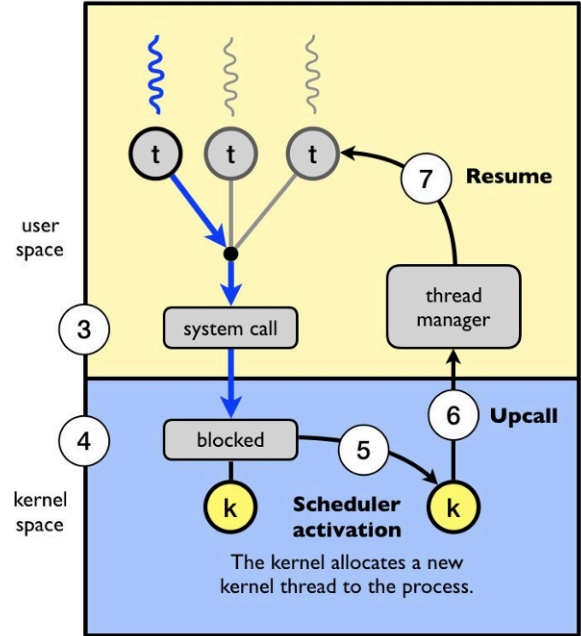
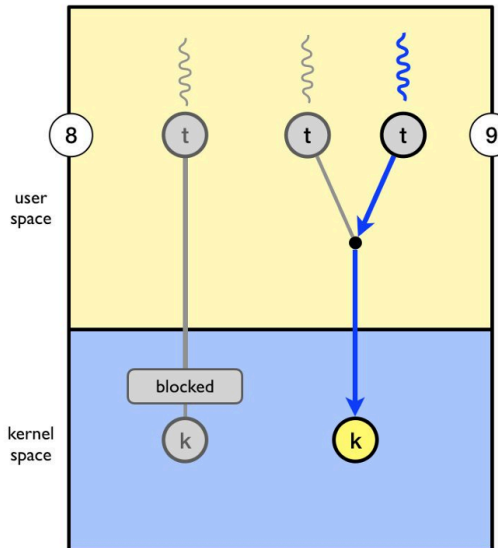
Schedule

Scheduler Activations

Bu sistemde işletim sistemi kernel seviyesinde bir block olduğunda bunu gizlemez ve kullanıcı seviyesindeki zamanlayıcıya haber verir.

Upcall Senaryosu

1. Elimizde 3 tane kullanıcı thread'i (User Thread) var ve bunlar kendilerine tahsis edilmiş 1 tane Kernel Thread (Sanal işlemci) üzerinde sırayla çalışıyorlar.
2. Thread gitti ve bloke edici bir sistem çağrısı yaptı (Örn: Diskten dosya okuma).
 - a. Normal sistemlerde ne olurdu? Kernel tüm süreci uyuturdu. Burada da Kernel o spesifik çağırın thread'i ve onun altındaki Kernel Thread'i mecburen bloke eder.
3. Kernel, bu sürecin tamamen felç olmasını engellemek için o sürece anında **yeni bir Kernel Thread (yeni bir sanal işlemci)** tahsis eder.



4. Kernel, yarattığı bu yeni Kernel Thread üzerinden Kullanıcı Seviyesindeki Kütüphaneye (Thread Manager) özel bir sinyal/mesaj gönderir . Der ki: *"Senin şu numaralı thread'in bloke oldu. Ama sana yepyeni bir Kernel Thread verdim."* İşletim sisteminden (aşağıdan), Kullanıcı Alanına (yukarıya) doğru yapılan bu özel bildirime **Upcall** denir.
5. Kullanıcı kütüphanesi (Thread Manager) Upcall mesajını alır almaz, bloke olan thread'i kenara ayırır (durumunu kaydeder). Hazırda bekleyen (ready) diğer thread'leri alır

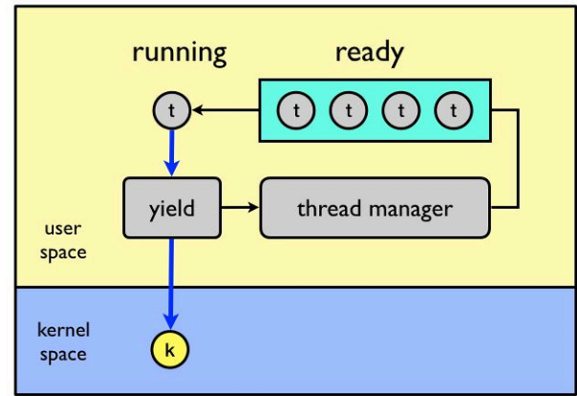
ve Kernel'ın verdiği bu *yeni* Kernel Thread üzerine koyarak tıkr tıkr çalıştırmaya devam eder.

Upcall Maliyeti: Kernel'ın normal işini gücünü bırakıp kullanıcı alanındaki kütüphaneye "Upcall" ile mesaj göndermesi ve kütüphanenin bunu işlemesi **çok ciddi bir zaman kaybı** yaratır. Performans kaybı yüksek olduğu için tasarımı çok zorlayıcıdır.

Scheduling Models at User-Level

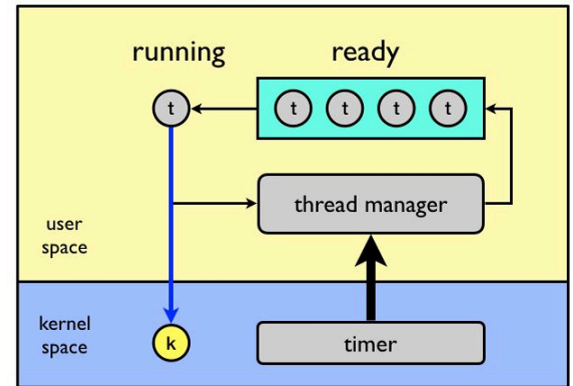
Cooperative (Yield)

- Çalışan bir thread işi bittiğinde veya yorulduğunda kendi rızasıyla `yield` (teslim ol) komutunu çağırır. Böylece kontrol Kütüphaneye (Thread Manager) geçer ve kütüphane Hazır (Ready) kuyruğundan sıradaki thread'i çalıştırır.
- Bencil bir thread sonsuz döngüye girip `yield` demezse tüm program kilitlenir.



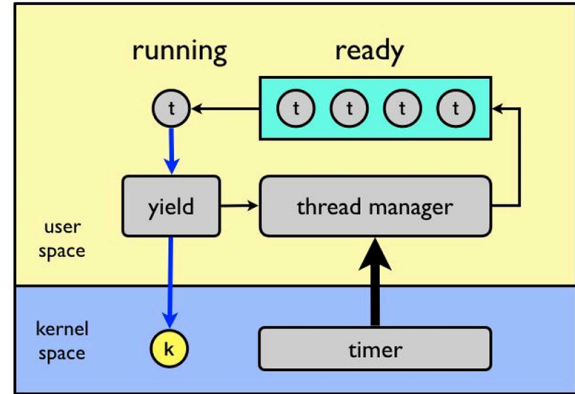
Preemptive

- Kütüphane thread'lerin inisiyatifine güvenmez. Sisteme donanımsal/yazılımsal bir zamanlayıcı (`timer`) kurar. Süre dolduğunda zamanlayıcı devreye girer, çalışan thread'i zorla durdurup kontrolü Kütüphaneye verir ve sıradaki thread çalıştırılır.



İkisi Birden

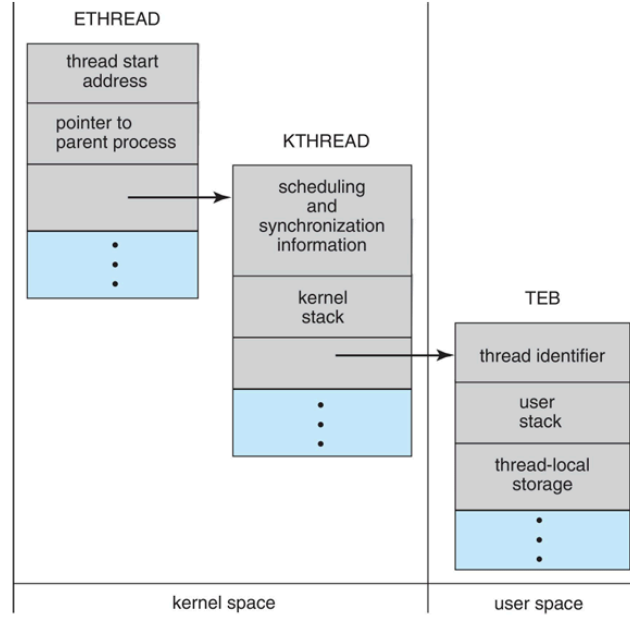
- Hem thread'lerin kendi rızasıyla işlemciyi bırakmasına izin verir hem de bencilleşenleri zorla kesmek için arkada bir `timer` bekletir.



OS Threads Data Structures

Windows Threads Data Structure

- Windows genelde one to one modeli kullanır.
- Windowsta bir thread yaratıldığında bellekte 3 zincirli devasa bir veri yapısı oluşur:
 - ETHREAD (Executive Thread Block):** Kernel space'te bulunur. Process'in kendine ve hemen altındaki KTHREAD'e işaret eden ana yönetim bloğudur.
 - KTHREAD (Kernel Thread Block):** Kernel space'tedir. İşletim sisteminin zamanlama bilgilerini tuttuğu yerdir. Ayrıca thread'in kernel stack'i de buradadır.
 - TEB (Thread Enviroment Block):** User space'te bulunur. Thread'in kimlik numarasını (ID), senin programında çalışan asıl **User Stack**'i ve o thread'e özel yerel değişken alanını (thread-local storage) içerir.



Linux Threads Data Structure

Linux, felsefesi gereği process'ler ve thread'ler arasında kesin bir çizgi çekmez. Linux'ta thread yoktur, hepsine task denir.

- Linuxta thread `clone()` adı verilen system call ile yaratılır.
- Peki `clone()` ve `fork()` arasında ne fark var?
 - `fork()` her şeyi birebir kopyalar ve iki ayrı dünya yaratır.
 - `clone()` ise **Bayraklar (Flags)** sayesinde neyin kopyalanıp neyin *paylaşılacağını* (share) milimetrik olarak seçmene izin verir.
- Önemli `clone()` flag'leri:
 - `CLONE_FS` : Dosya sistemi bilgileri paylaşılır.
 - `CLONE_VM` : Sanal bellek (Virtual Memory) alanı paylaşılır (İşte bu, iki süreci birer "thread" haline getiren asıl bayraktır).
 - `CLONE_SIGHAND` : Sinyal işleyicileri ortak kullanılır.
 - `CLONE_FILES` : Açık dosyalar (open files) ortak kullanılır.

Implicit (Örtük) Threading

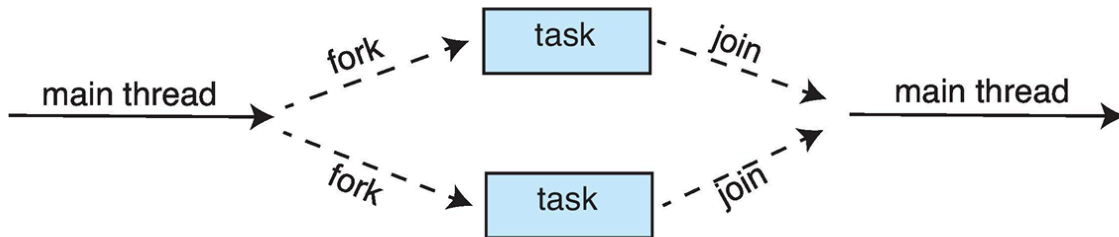
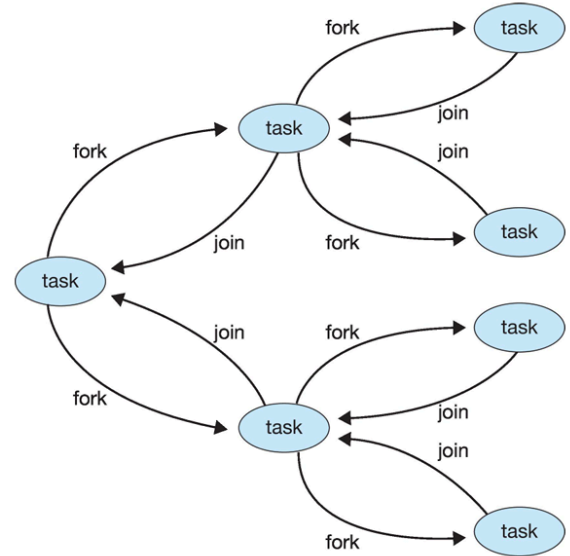
Derleyicilerin ve arka plandaki kütüphanelerin iş parçacıklarını bizim yerimize nasıl yönettiğine dair endüstride standart olmuş 5 yöntem şunlardır:

Thread Pools

- Program başlarken belirli sayıda thread yaratılır ve bunlar bir "havuzda" uyur halde bekletilir.
- Yeni bir görev (task) geldiğinde sıfırdan thread yaratmak yerine, havuzdaki uyanık bir thread'e bu görev verilir. İş bitince thread ölmez, havuza geri döner.

Fork-Join Paralelliği

- Böl ve yönet (Divide and conquer) mantığıyla çalışır.
- Ana thread, büyük bir problemi küçük alt problemlere böler ve bunlar için yeni görevler yaratır (fork).
- Alt görevler işini bitirdiğinde sonuçlar geriye doğru birleştirilir (join) ve nihai sonuca ulaşılır.



OpenMP

- C, C++ ve Fortran dilleri için geliştirilmiş derleyici direktifleridir. Kodun içine sadece `#pragma omp parallel` yazarsın.
- Derleyici, sistemdeki çekirdek sayısı kadar thread'i otomatik olarak yaratır ve altındaki kod bloğunu veya `for` döngüsünü o çekirdeklere paylaştırır.

Grand Central Dispatch / GCD

- Sen sadece çalışmasını istediğin kod bloklarını `^{}` bir kuyruğa (dispatch queue) atarsın.
- Seri veya eşzamanlı çalışabilen bu kuyrukları işletim sistemi arka plandaki kendi thread havuzuna kendi bildiği gibi dağıtır.

Intel Threading Build Blocks / TBB

- C++ için tasarlanmış, döngüleri ve algoritmaları çalışma zamanında (runtime) paralel hale getiren bir şablondur.

Threading Issues

Çok sayıda thread'in aynı evin (sürecin) içinde yaşaması bazı kritik mantık hatalarına yol açar.

1. `fork()` ve `exec()` İkilemi:

- Eğer 10 thread'i olan bir programda, sadece bir tane thread `fork()` çağırıp süreci klonlamak isterse ne olur? Yeni kopyada sadece o tek thread mi olacak, yoksa 10 thread birden mi kopyalanacak? UNIX sistemleri bu yüzden `fork()` komutunun iki farklı versiyonunu sunar .
- `exec()` çağrıldığında ise durum nettir: Yeni program hafızayı tamamen ezeceği için içerideki *tüm* thread'ler anında yok edilir.

2. Sinyaller Kime Gidecek?

- İşletim sisteminden bir sinyal geldi (Örneğin Ctrl+C ile kapatma sinyali). Tek thread'li sistemlerde hedef bellidir. Peki çoklu thread'de bu sinyali kim karşılayacak? İşletim sistemleri şu 4 seçenekten birini uygular:

1. Sinyali sadece onunla ilgili olan spesifik thread'e gönder.

2. Sinyali istisnasız tüm thread'lere gönder.
3. Sinyali sadece önceden belirlenmiş bazı thread'lere gönder.
4. Sürecin içindeki tüm sinyalleri karşılaması için özel, tek bir thread ata.

3. Thread İptal Etme:

- Bir thread'in işini bitirmeden önce dışarıdan zorla sonlandırılması (iptal edilmesi) gerekirse iki yol vardır:
 - **Asenkron İptal (Asynchronous):** Hedef thread nerede olduğuna veya ne yaptığına bakılmaksızın **anında, acımasızca öldürülür**. Eğer o sırada önemli bir veriyi güncelliyorsa veri bozulur!
 - **Ertelenmiş İptal (Deferred):** Çok daha güvenlidir. Sistem thread'e "Seni iptal etmek istiyorlar" diye bir bayrak (flag) çeker. Thread işine devam eder ama periyodik olarak o bayrağa bakar. Uygun bir duruma geldiğinde kendi temizliğini yapıp kendi rızasıyla kapanır.

4. Thread'e Özel Depolama (TLS)

- Hep "Thread'ler aynı hafızayı (Data bölümünü) paylaşır" demiştik. Ama bazen bir thread'in sadece kendine özel olan ama fonksiyonlar arasında silinmeyen global benzeri bir veriye ihtiyacı olur.
- **TLS**, her thread'in sadece kendi erişebildiği ama süreç var oldukça hayatta kalan özel bir bellek alanıdır. Yerel değişkenlerden farkı; yerel değişkenler fonksiyon bitince silinir, TLS ise thread yaşayana kadar var olmaya devam eder .